Agile Software Development

Week 4

Chanduni Gamage
Lecturer (Probationary)
Dept. of ICT
Faculty of Technology
University of Ruhuna

Agile Methods

- Dissatisfaction with the overheads involved in software design methods of the 1980s and 1990s led to the creation of agile methods. These methods:
  - Focus on the code rather than the design
  - Are based on an iterative approach to software development
  - Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.
- The aim of agile methods is to reduce overheads in the software process (e.g. by limiting documentation) and to be able to respond quickly to changing requirements without excessive rework.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

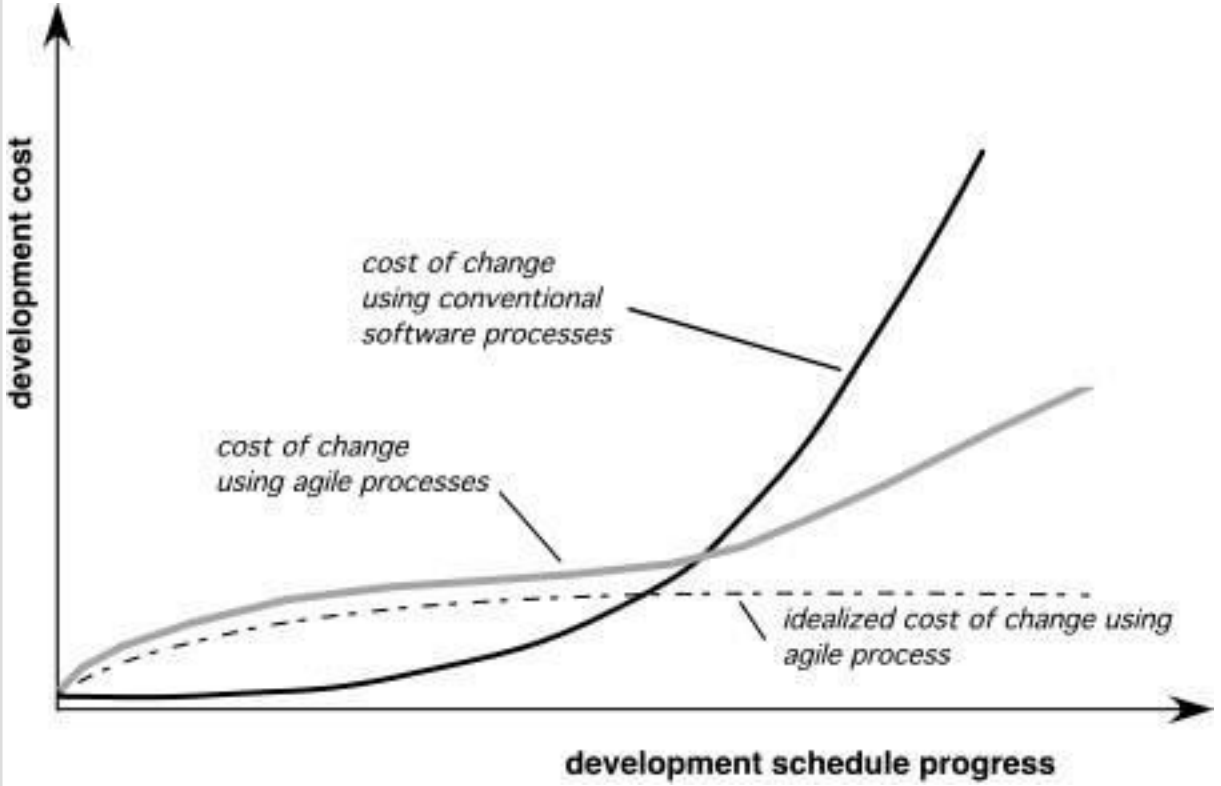
What is “Agility”?

- Effective (rapid and adaptive) response to change
- Effective communication among all stakeholders
- Drawing the customer onto the team
- Organizing a team so that it is in control of the work performed

Yielding …

- Rapid, incremental delivery of software

Agility and the Cost of Change



The Manifesto for Agile Software Development

"We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- ***Individuals and interactions* over processes and tools**
- ***Working software* over comprehensive documentation**
- ***Customer collaboration* over contract negotiation**
- ***Responding to change* over following a plan**

That is, while there is value in the items on the right, we value the items on the left more."

Kent Beck et al

What is Agile Software Development

- Easily moved, light nimble, active software process
  - Latin word agilis, which means “easily moved, light, nimble, active”
  - Agile Software development – defined as easily moved, light, nimble, and active software processes
- Fitting the process to the project
- Avoidance of things that waste time
- Fitting the process…Not project to the process

What is Agile?

An iterative and incremental (evolutionary) approach performed in a highly collaborative manner with **just the right amount of ceremony** to produce high quality software in a cost effective and timely manner which meets the changing needs of its stakeholders.



How Agile is Different?

- Focus on collaboration:
  - Less paperwork and more conversation
  - Stakeholders actively involved
- Focus on working software:
  - Greater feedback makes agile projects easier to manage
  - Less documentation is required
  - Less bureaucracy
- Agilists are generalizing specialists:
  - Specialists find it difficult at first to fit into the team
- Agile is based on practice, not theory:
  - This is a significant change from traditional
  - You need to see how agile works in practice to truly understand it

The Principles of Agile Methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

Myth busters

Myths

- No Documentation
- Undisciplined
- No Planning
- Not Predictable
- Silver Bullet
- Not Fixed Price

Reality

- Agile Documentation
- Requires great discipline
- Just-in-time(JIT) planning
- Far more predictable
- It requires skilled people
- Agile provides stakeholders control over the budget, schedule, and scope

Agile Method Applicability

- Product development where a software company is developing a small or medium-sized product for sale.
- Custom system development within an organization, where there is a clear commitment from the customer to become involved in the development process and where there are not a lot of external rules and regulations that affect the software.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Problems with Agile Methods

- It can be difficult to keep the interest of customers who are involved in the process.
- Team members may be unsuited to the intense involvement that characterizes agile methods.
- Prioritizing changes can be difficult where there are multiple stakeholders.
- Maintaining simplicity requires extra work.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Agile Methods and Software Maintenance

- Most organizations spend more on maintaining existing software than they do on new software development. So, if agile methods are to be successful, they have to support maintenance as well as original development.
- Two key issues:
  - Are systems that are developed using an agile approach maintainable, given the emphasis in the development process on minimizing formal documentation?
  - Can agile methods be used effectively for evolving a System in response to customer change requests?
- Problems may arise if the original development team cannot be maintained.

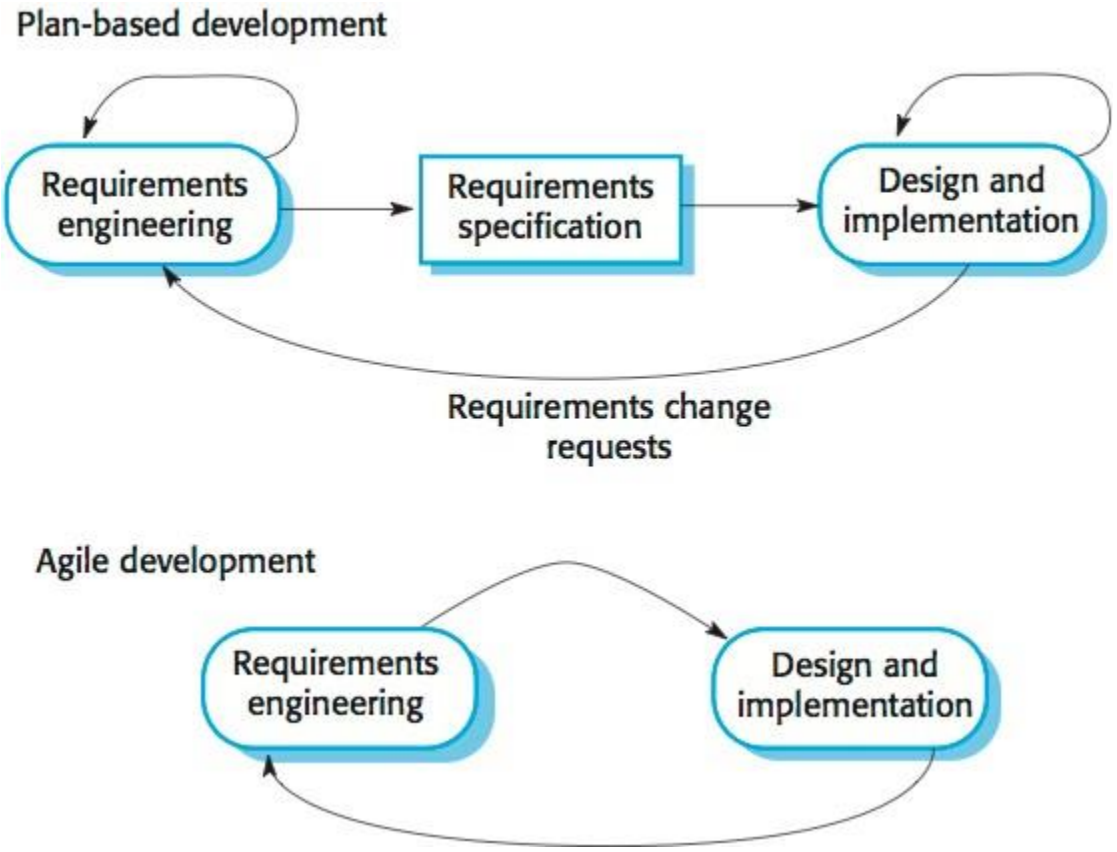
Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Plan-driven and Agile Development

- Plan-driven development
  - A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance.
  - Not necessarily waterfall model – plan-driven, incremental development is possible
  - Iteration occurs within activities.
- Agile development
  - Specification, design, implementation, and testing are inter-leaved and the outputs from the development process are decided through a process of negotiation during the software development process.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Plan-driven and Agile Specification



Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Technical, Human, Organizational Issues

- Most projects include elements of plan-driven and agile processes. Deciding on the balance depends on:
  - Is it important to have a very detailed specification and design before moving to implementation? If so, you probably need to use a plan-driven approach.
  - Is an incremental delivery strategy, where you deliver the software to customers and get rapid feedback from them, realistic? If so, consider using agile methods.
  - How large is the system that is being developed? Agile methods are most effective when the system can be developed with a small co-located team that can communicate informally. This may not be possible for large systems that require larger development teams so a plan-driven approach may have to be used.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Technical, Human, Organizational Issues

- What type of system is being developed?
  - Plan-driven approaches may be required for systems that require a lot of analysis before implementation (e.g. real-time systems with complex timing requirements).
- What is the expected system lifetime?
  - Long-lifetime systems may require more design documentation to communicate the original intentions of the system developers to the support team.
- What technologies are available to support system development?
  - Agile methods rely on good tools to keep track of an evolving design

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Technical, Human, Organizational Issues

- How is the development team organized?
  - If the development team is distributed or if part of the development is being outsourced, then you may need to develop design documents to communicate across the development teams.
- Are there cultural or organizational issues that may affect the system development?
  - Traditional engineering organizations have a culture of plan-based development, as this is the norm in engineering.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Technical, Human, Organizational Issues

- How good are the designers and programmers in the development team?
  - It is sometimes argued that agile methods require higher skill levels than plan-based approaches in which programmers simply translate a detailed design into code
- Is the system subject to external regulation?
  - If a system has to be approved by an external regulator (e.g. the FAA approves software that is critical to the operation of an aircraft) then you will probably be required to produce detailed documentation as part of the system safety case.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Extreme Programming (XP)

- The most widely used agile process, originally proposed by Kent Beck
- XP Planning
  - Begins with the creation of "user stories"
  - The Agile team assesses each story and assigns a cost
  - Stories are grouped to form a deliverable increment
  - A commitment is made on the delivery date
  - After the first increment "project velocity" is used to help define subsequent delivery dates for other increments

Requirements Scenarios

- In XP, a customer or user is part of the XP team and is responsible for making decisions on requirements.
- User requirements are expressed as **scenarios** or **user stories**.
- These are written on cards and the development team breaks them down into implementation tasks. These tasks are the basis of schedule and cost estimates.
- The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

A ‘Prescribing Medication’ Story

Prescribing medication

The record of the patient must be open for input. Click on the medication field and select either ‘current medication’, ‘new medication’ or ‘formulary’.

If you select ‘current medication’, you will be asked to check the dose; If you wish to change the dose, enter the new dose then confirm the prescription.

If you choose, ‘new medication’, the system assumes that you know which medication you wish to prescribe. Type the first few letters of the drug name. You will then see a list of possible drugs starting with these letters. Choose the required medication. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

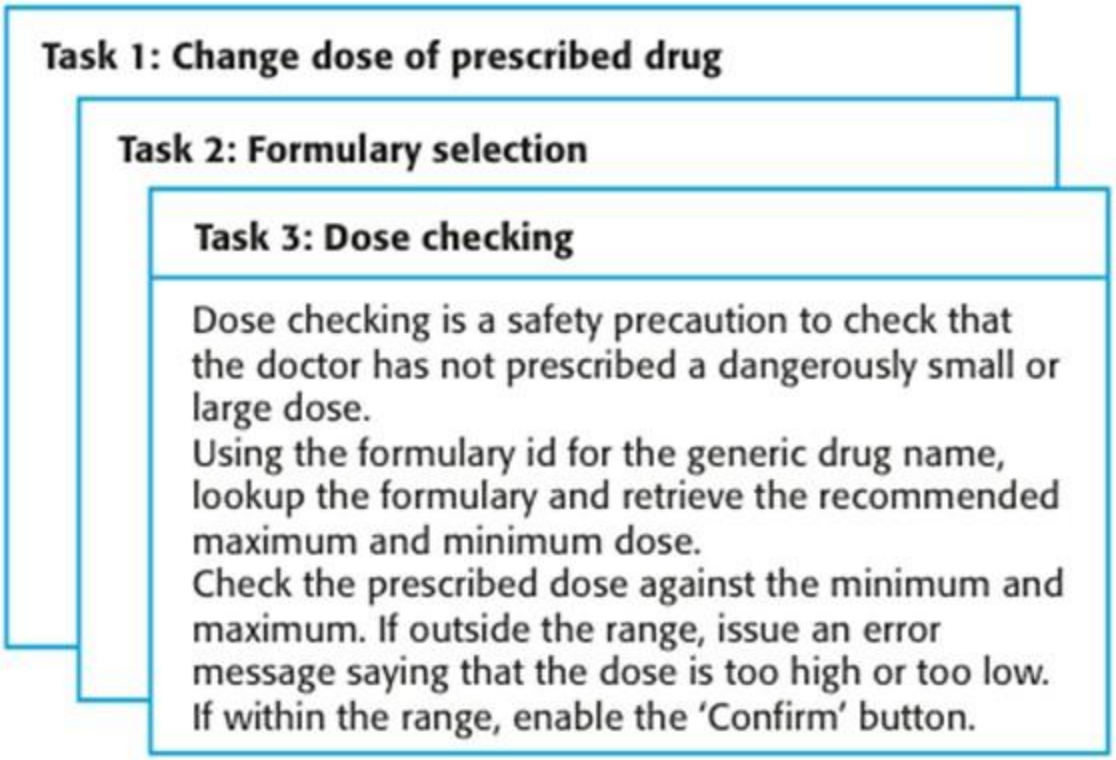
If you choose ‘formulary’, you will be presented with a search box for the approved formulary. Search for the drug required then select it. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

In all cases, the system will check that the dose is within the approved range and will ask you to change it if it is outside the range of recommended doses.

After you have confirmed the prescription, it will be displayed for checking. Either click ‘OK’ or ‘Change’. If you click ‘OK’, your prescription will be recorded on the audit database. If you click ‘Change’, you reenter the ‘Prescribing medication’ process.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Examples of Task Cards for Prescribing Medication



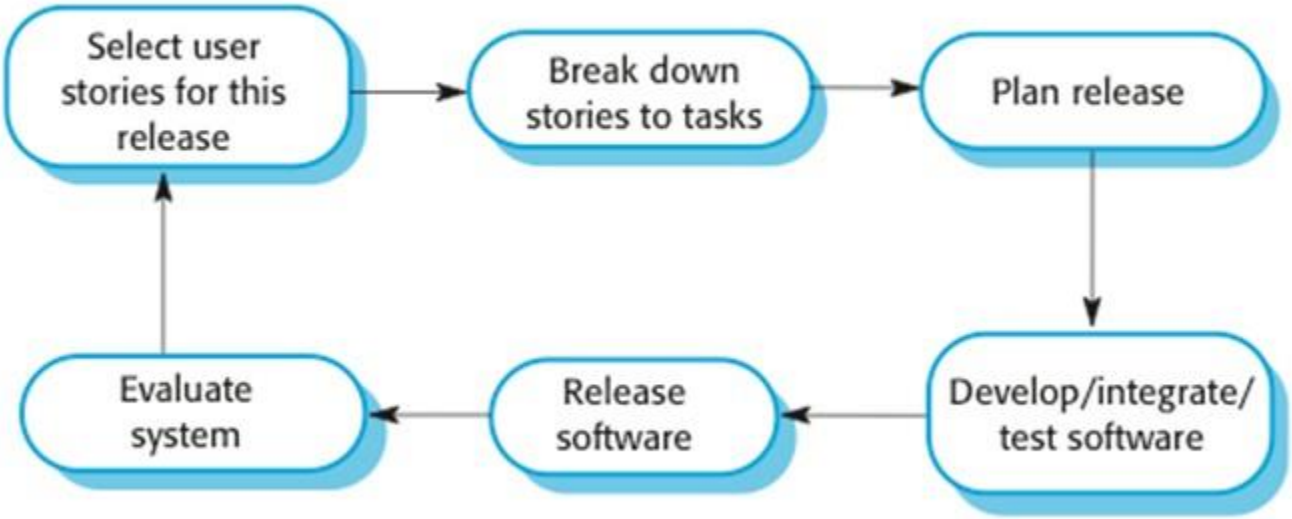
Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

XP and Agile Principles

- Incremental development is supported through small, frequent system releases.
- Customer involvement means full-time customer engagement with the team.
- People process through pair programming, collective ownership, and a process that avoids long working hours.
- Change supported through regular system releases.
- Maintaining simplicity through constant refactoring of code.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

The Extreme Programming Release Cycle



Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Extreme Programming Practices (a)

Principle or practice	Description
Incremental planning	Requirements are recorded on story cards and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development 'Tasks'.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Simple design	Enough design is carried out to meet the current requirements and no more.
Test-first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.

Extreme Programming Practices (b)

Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Sustainable pace	Large amounts of overtime are not considered acceptable as the net effect is often to reduce code quality and medium term productivity
On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Pair Programming

- In XP, programmers work in pairs, sitting together to develop code.
- This helps develop common ownership of code and spreads knowledge across the team.
- It serves as an informal review process as each line of code is looked at by more than 1 person.
- It encourages refactoring as the whole team can benefit from this.
- In pair programming, programmers sit together at the same workstation to develop the software.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Pair Programming

- Pairs are created dynamically so that all team members work with each other during the development process.
- The sharing of knowledge that happens during pair programming is very important as it reduces the overall risks to a project when team members leave.
- Pair programming is not necessarily inefficient and there is evidence that a pair working together is more efficient than 2 programmers working separately.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Test automation

- Test automation means that tests are written as executable components before the task is implemented
  - These testing components should be stand-alone, should simulate the submission of input to be tested, and should check that the result meets the output specification. An automated test framework (e.g. Junit) is a system that makes it easy to write executable tests.
- As testing is automated, there is always a set of tests that can be quickly and easily executed
  - Whenever any functionality is added to the system, the tests can be run and problems that the new code has introduced can be caught immediately.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Refactoring

- The programming team looks for possible software improvements and makes these improvements even where there is no immediate need for them.
- This improves the understandability of the software.
- Changes are easier to make because the code is Well-structured and clear.
- However, some changes require architecture refactoring, and this is much more expensive.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Examples of Refactoring

- Re-organization of a class hierarchy to remove duplicate code.
- Tidying up and renaming attributes and methods to make them easier to understand.
- The replacement of inline code with calls to methods that have been included in a program library.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Scrum

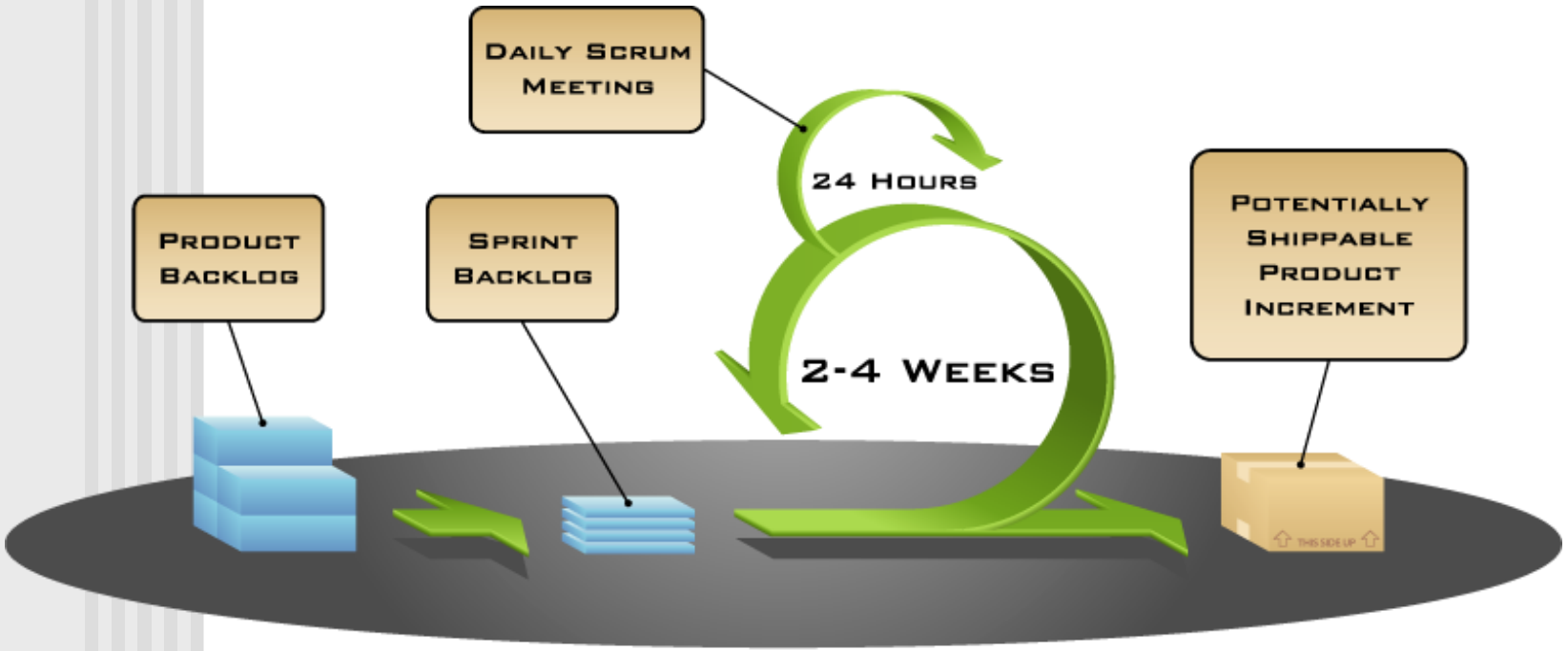
- The Scrum approach is a general agile method but its focus is on managing iterative development rather than specific agile practices.
- There are three phases in Scrum.
  - The initial phase is an outline planning phase where you establish the general objectives for the project and design the software architecture.
  - This is followed by a series of sprint cycles, where each cycle develops an increment of the system.
  - The project closure phase wraps up the project, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Scrum

- Originally proposed by Schwaber and Beedle
- Scrum—distinguishing features
  - Development work is partitioned into "packets"
  - Testing and documentation are on-going as the product is constructed
  - Work occurs in "sprints" and is derived from a "backlog" of existing requirements
  - Meetings are very short and sometimes conducted without chairs
  - "demos" are delivered to the customer with the time-box allocated

The Scrum Process



The Sprint Cycle

- Sprints are fixed length, normally 2–4 weeks.
- The starting point for planning is the product backlog, which is the list of work to be done on the project.
- The selection phase involves all of the project team who work with the customer to select the features and functionality to be developed during the sprint.

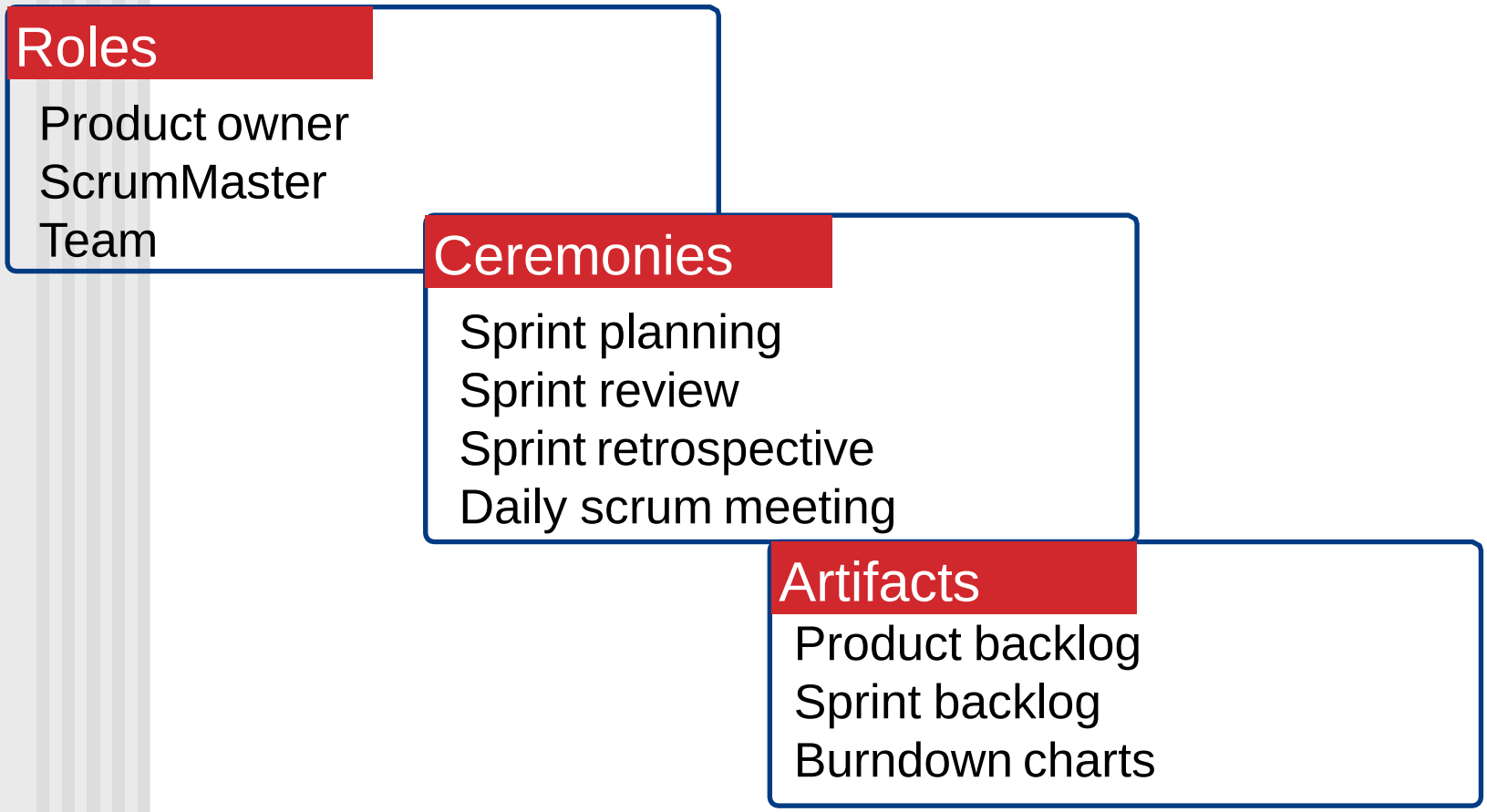
Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

The Sprint Cycle

- Once these are agreed, the team organize themselves to develop the software. During this stage the team is isolated from the customer and the organization, with all communications channelled through the so-called ‘Scrum master’.
- The role of the Scrum master is to protect the development team from external distractions.
- At the end of the sprint, the work done is reviewed and presented to stakeholders. The next sprint cycle then begins.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

The SCRUM Framework



Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

The SCRUM Framework

Product owner

- Define the features of the product
- Decide on release date and content
- Be responsible for the profitability of the product (ROI)
- Prioritize features according to market value
- Adjust features and priority every iteration, as needed
- Accept or reject work results

ScrumMaster

- Represents management to the project
- Responsible for enacting Scrum values and practices
- Removes impediments
- Ensure that the team is fully functional and productive
- Enable close cooperation across all roles and functions
- Shield the team from external interferences

Team

- Typically 5-9 people
- Cross-functional:Programmers, testers, user experience designers, etc.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Teamwork in Scrum

- The ‘Scrum master’ is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside of the team.
- The whole team attends short daily meetings where all team members share information, describe their progress since the last meeting, problems that have arisen and what is planned for the following day.
  - This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Scrum Benefits

- ✓ The product is broken down into a set of manageable and understandable chunks.
- ✓ Unstable requirements do not hold up progress.
- ✓ The whole team have visibility of everything and consequently team communication is improved.
- ✓ Customers see on-time delivery of increments and gain feedback on how the product works.
- ✓ Trust between customers and developers is established and a positive culture is created in which everyone expects the project to succeed.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Scaling Agile Methods

- Agile methods have proved to be successful for small and medium sized projects that can be developed by a small co-located team.
- It is sometimes argued that the success of these methods comes because of improved communications which is possible when everyone is working together.
- Scaling up agile methods involves changing these to cope with larger, longer projects where there are multiple development teams, perhaps working in different locations.

Ian Sommerville - Software Engineering - 9th Edition, © Pearson Education

Thank You!